QA Engineer Backend Technical Challenge

Dear candidate,

Thank you again for your application for the position as a Quality Assurance Engineer (m/f/d).

This challenge is designed to help us understand how you approach backend test automation, API validation, and test design.

Automation Test Challenge

As a Senior QA, one of your key missions is to ensure the reliability of our internal services. In this challenge, you will build a small, automated testing suite to monitor Pull Requests for a specific product.

These requirements apply to both tasks below:

- **Language:** Use the language you are most comfortable with (TypeScript or Go preferred).
- **Code Structure:** Organize your code in a way that keeps API communication, business logic, and test assertions clearly separated.
- **Observability:** Your tests should provide useful output so failures can be easily understood in a CI environment.
- **README** Please include a short README explaining:
    - Your architectural decisions
    - Any assumptions made
    - How you approached pagination and validation

We value solutions that are maintainable, reusable, and easy to understand. Assume that this testing logic may need to be reused across multiple projects..

Part 1: The API Integration (Live Data)

Target: https://api.github.com/repos/appwrite/appwrite/pulls

Task: Create an automated test that fetches the list of **open** Pull Requests.

> **The Challenge**: Your test must ensure it captures all currently **“open”** Pull Requests across the entire dataset to provide an accurate total count.
>
> **Objective:** Validate the structure of the API response using a schema validator (for example Zod, Joi, or JSON Schema) to ensure the response format is as expected.
>
> **Business Rule:** Only pull requests in an OPEN state should be included in the final count. Pull requests marked as draft must not be counted.

Part 2: Business Rule Validation (Mocked Scenario)

Imagine our system consumes a middleware service that aggregates pull request data from multiple sources. For this exercise, use the following JSON payload as the service response and validate that it satisfies the expected business rules.

Input JSON to validate:

```JSON
{
  "product_id": "appwrite-001",
  "total_open_prs": 1,
  "last_updated": "2024-03-20T15:30:00Z",
  "pull_requests": [
    {
      "id": 1024,
      "title": "feat: add SQS support to testing commons",
      "author": { "username": "nej.jissard", "role": "admin" },
      "status": "OPEN",
      "labels": ["backend", "high-priority"],
      "meta": { "is_draft": false, "review_comments": 5 }
    }
  ]
}
```

Task: Validate the following conditions based on the provided response:

> **Integrity:** The number of items in the pull_requests array must match total_open_prs.
>
> **Business Rule:** If a pull request has the label "high-priority", it must not be marked as draft. If this rule is violated, the test should fail with a clear error message.